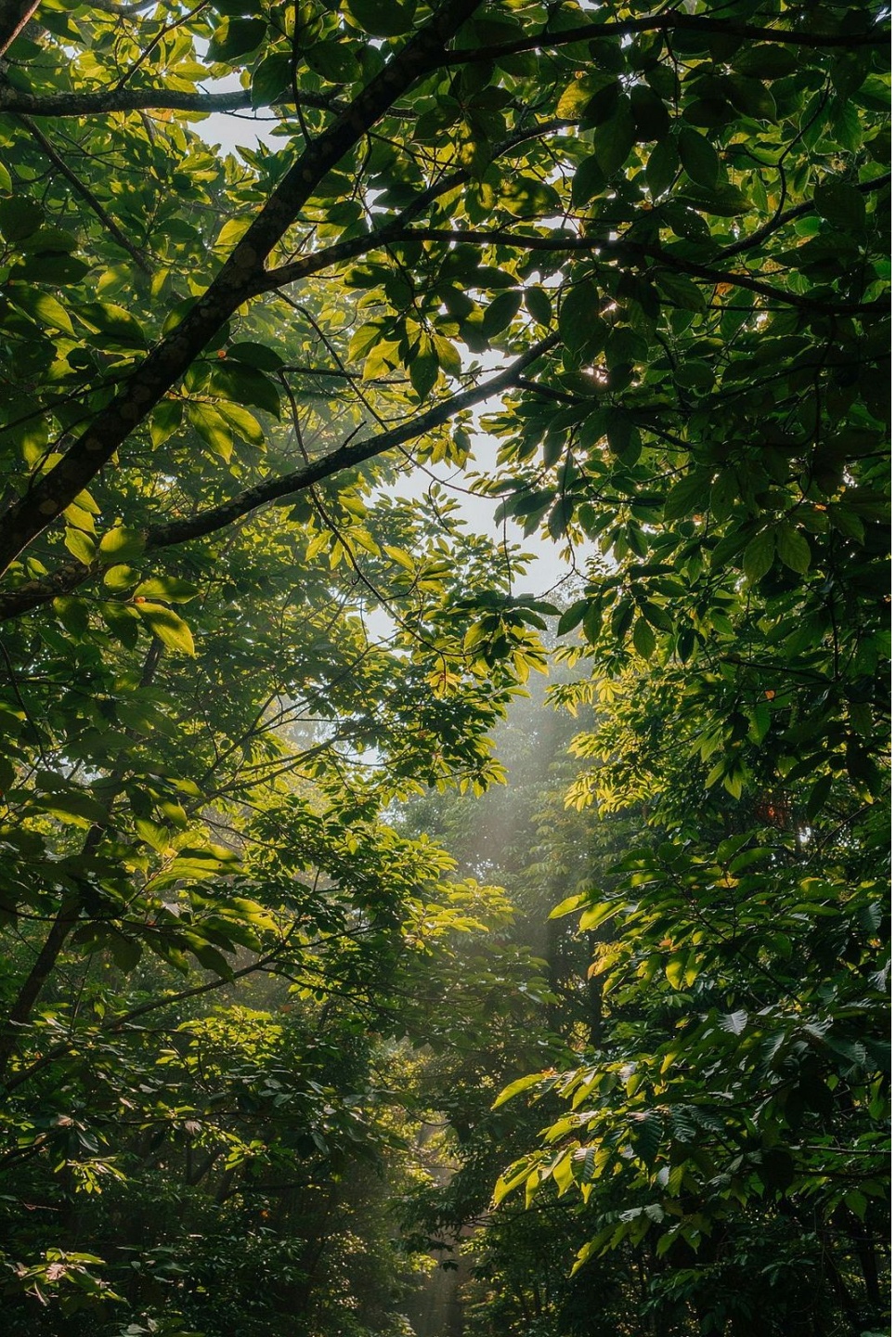


Sprint 8 · Duelo de modelos · S08.2

Ensembles: Random Forest y boosting

Un solo árbol es inestable. Muchos árboles juntos son más precisos y más estables, a cambio de explicabilidad y coste.

Dos estrategias para juntar árboles

Bagging → Random Forest

Muchos árboles independientes, cada uno con una muestra con reemplazo y variables al azar; se promedian.

Boosting

Árboles pequeños en cadena: cada uno corrige los errores del anterior, sumado con un paso pequeño (learning_rate).

Si todos los árboles fueran iguales, promediar no serviría: por eso se introduce aleatoriedad (muestras y variables).

Random Forest (cuaderno S08_02)

```
bosque = RandomForestClassifier(n_estimators=300, min_samples_leaf=5,
                                random_state=SEMILLA, n_jobs=2).fit(Xtr, y_train)
auc_test_bosque = roc_auc_score(y_test, bosque.predict_proba(Xte)[:, 1])   # Ejercicio 1
```

- **n_estimators:** más árboles casi nunca perjudica; se estabiliza (de 100 a 400 árboles la mejora es mínima: Ejercicio 2).
- **min_samples_leaf** y **max_depth** controlan el sobreajuste de cada árbol; **max_features** = variables por pregunta.
- La puntuación OOB (las reservas que cada árbol no vio) da una validación gratis.



Importancia: impureza frente a permutación

	Impureza (feature_importances_)	Permutación
Cómo	lo que reduce Gini en el entrenamiento	se baraja una columna del test y se mide cuánto cae el AUC
Pega	sesgada a numéricas con muchos valores; calculada sobre train	más lenta
Ventaja	rápida	mide el efecto real sobre la predicción

Ejercicio 3: variable_top y n_irrelevantes (importancia < 0,002).

El bosque gana rendimiento y pierde explicabilidad: 300 árboles no se leen. Además cuesta más tiempo y memoria. Lo medimos frente al árbol y la logística en el mini-reto (duelo).

Boosting y el peligro de pasarse (cuaderno S08_03)

1. Un primer árbol muy pequeño predice.
2. El siguiente se entrena para predecir los errores (residuos).
3. Se suma multiplicado por learning_rate. Se repite cientos de veces.

```
mejor_n_arboles = int(np.argmax(auc_val_por_arbol)) + 1        # Ejercicio 1
caida_final = max(auc_val_por_arbol) - auc_val_por_arbol[-1]
```

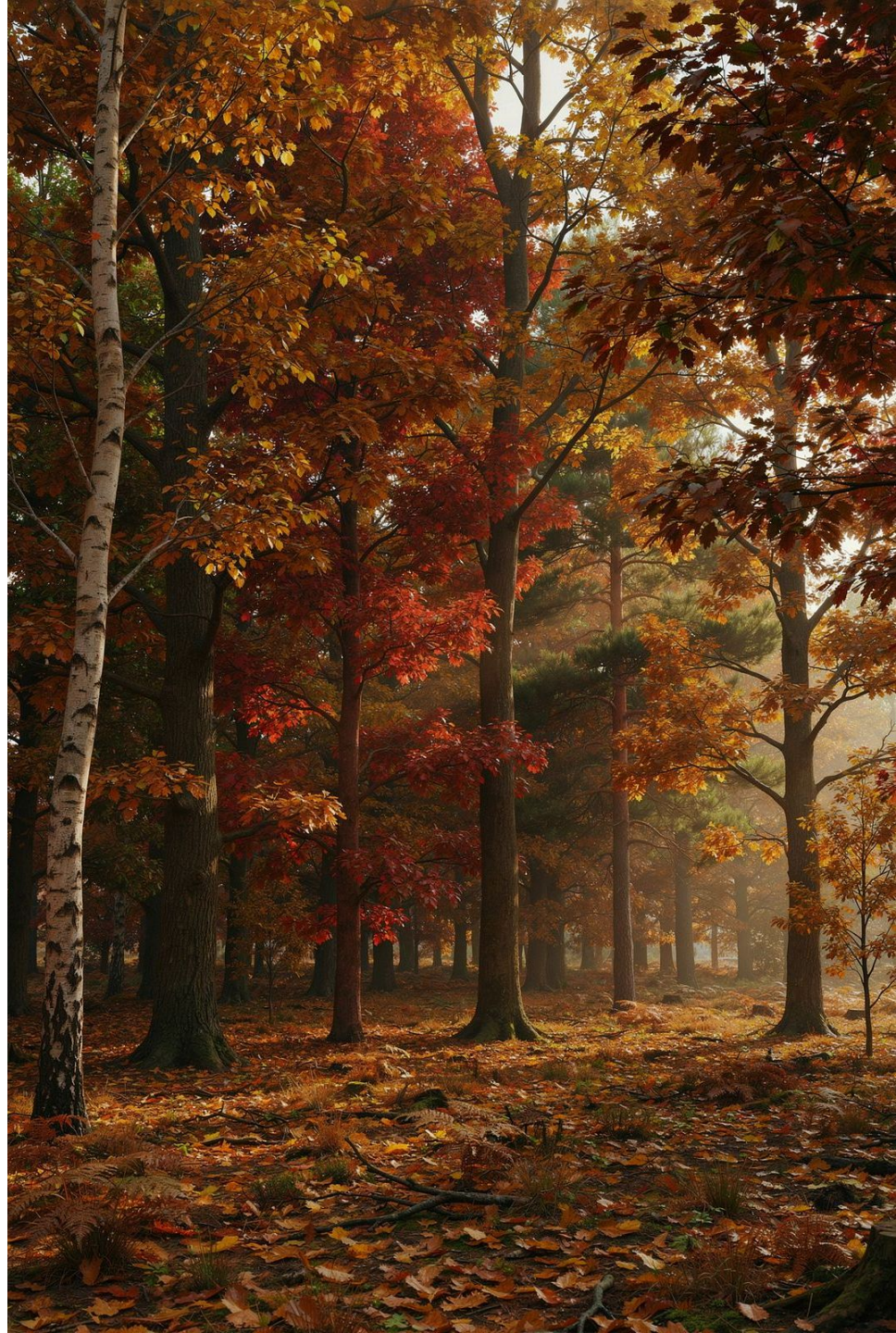
Con demasiados árboles, el AUC de validación sube y luego baja: sobreajuste. Solución: parada temprana (máximo alto de árboles y se detiene si la validación no mejora N rondas), con tasa baja (0,03–0,1).

XGBoost y LightGBM

	XGBoost	LightGBM
Crecimiento	por niveles (max_depth)	por hojas (num_leaves)
Control de hojas	min_child_weight	min_child_samples
Parada temprana	early_stopping_rounds + eval_set	callbacks + eval_set
Importancias	por ganancia	por ganancia

```
modelo_xgb.fit(Xfit, yfit, eval_set=[(Xval, yval)], verbose=False)
modelo_lgb.fit(Xfit, yfit, eval_set=[(Xval, yval)], eval_metric="auc", callbacks=[...])
```

Mini-reto: tabla con auc_test, arboles y segundos_ajuste de sklearn_gb, xgboost y lightgbm.



Primeros modelos en el leaderboard (S08_06)

```
registrar(evaluar_modelo("baseline", "referencia",
                         hacer_pipe(DummyClassifier(strategy="prior")), 5, notas))   # Ejercicio 1
registrar(evaluar_modelo("random_forest", "ensemble_bagging", hacer_pipe(...), 2, notas))
```

- evaluar_modelo mide AUC y AP (validación cruzada), tiempo de ajuste, tiempo de predicción por 1000 reservas y tamaño en KB; la explicabilidad (1–5) la pones tú.
- Hoy: baseline, logística, árbol y bosque (pasos 1–2 del reto). SVM y búsqueda, en la siguiente sesión.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

- **CUADERNO · Random Forest: bagging, OOB e importancia por permutación** (sprints/S08/notebooks/S08_02_random_forest) — 40 min.
- **CUADERNO · Gradient Boosting, XGBoost y LightGBM** (sprints/S08/notebooks/S08_03_boosting_xgboost_lightgbm) — 45 min.
- **RETO · Primeros modelos en el leaderboard (baseline, logística, árbol y bosque; pasos 1–2)** (sprints/S08/notebooks/S08_06_leaderboard_plantilla) — 25 min.